

RetailFlow security, data and delivery remediation brief

A detailed, source-checked implementation brief for Antigravity. Each finding includes the current state, the required fix, proof of completion, and a ready-to-paste build prompt.

Scope and working rule

Apply these changes in a branch and deploy database changes to staging first. Create new timestamped Supabase migrations instead of editing an already-applied migration. Do not expose secrets, service-role keys, customer phone numbers, or raw Gemini errors in the client.

Recommended delivery order

- 1. Secure AI and WhatsApp Edge Functions before adding new AI or messaging features.
- 2. Repair tenant RLS and prove isolation with two-account tests before touching onboarding or uploads.
- 3. Minimise AI data and add consent, then deliver the quotation lifecycle as persisted business data.
- 4. Replace dashboard fixtures with queries and finish or hide incomplete controls.
- 5. Standardise statuses, authentication feedback, encoding, currency, and regression tests.

Finding index

Priority	Current state	Finding
Critical	Confirmed	Unauthenticated AI and WhatsApp functions
Critical	Confirmed	RLS enabled on shops with no shops policies
High	Confirmed	Dashboard metrics and follow-ups are hard-coded
High	Partial	Quote UI exists but no persisted sales lifecycle
High	Mixed	Some controls work; More and product action remain incomplete
Medium	Confirmed	Customer status vocabulary is inconsistent
Medium	Partial	Login misses recovery and visibility controls
Medium	Confirmed	Mojibake and INR labels need normalisation
Medium	Confirmed	AI prompt includes customer phone numbers and unbounded images

Important reconciliation notes from the current RetailFlow copy

- The current QuoteBuilder is a polished local-state screen. It is not yet a persisted quotation workflow and does not create a PDF, approval, order, or payment record.
- Product filters and customer status filters are already functional in the refreshed UI. Do not replace them with placeholder code. The unfinished areas are the More setting rows and the visible product three-dot action, plus any unused favourite contract.
- The current login success message is already green, not red. Keep that treatment while adding missing recovery, password visibility, and better OAuth feedback.

Require authentication and rate limits for AI and WhatsApp

CRITICAL

CONFIRMED

Current code evidence

supabase/config.toml sets `verify_jwt = false` for `gemini-proxy` and `whatsapp-sender`. Both Edge Functions accept a JSON request immediately; neither resolves the caller or their shop. The WhatsApp function accepts any phone and message, and the Gemini function accepts arbitrary actions, prompts, history and image parts.

Why this matters

Anyone who can reach these endpoints can consume Gemini credits or trigger WhatsApp sends. This is both a cost-control and abuse risk. Client-side checks do not protect Edge Functions because callers can invoke their URLs directly.

Implementation contract

- Set `verify_jwt = true` for both functions and reject a missing or invalid Authorization bearer token with HTTP 401 before parsing or processing work.
- Inside each function, use the caller JWT to resolve `auth.getUser()`, then read `profiles.shop_id` server-side. Never trust a `shop_id`, phone number, role, price, or customer object from the browser.
- For WhatsApp, accept a `customer_id` and a message template/body. Fetch the customer by `customer_id` plus the caller `shop_id` and use the stored phone only after ownership is confirmed.
- Validate every request with an allow-list schema: permitted Gemini actions, maximum text/history lengths, maximum image count, MIME allow-list, and a bounded decoded body size. Return safe error messages only.
- Add atomic per-user and per-shop rate limiting in Postgres or a purpose-built rate-limit service. Use stricter quotas for image generation and WhatsApp; return 429 with `Retry-After` when exceeded.
- Restrict CORS to configured application origins in production, record minimal audit events, and never log prompt payloads, access tokens, API keys, or phone numbers.

Acceptance checks

- No Authorization header returns 401 for both function URLs.
- A valid user from shop A cannot send to a customer in shop B or access its data.
- Requests above each configured rate cap receive 429, while a request below the cap succeeds.
- Malformed actions, oversized history, image MIME mismatches and too-large payloads receive 400 before Gemini or Meta is called.
- Function tests prove no WhatsApp or Gemini provider call occurs for rejected requests.

READY-TO-PASTE ANTIGRAVITY PROMPT

Work in the RetailFlow Supabase Edge Functions and config. Secure `gemini-proxy` and `whatsapp-sender` end to end. Enable JWT verification in config, authenticate the bearer token in the handler, derive user and shop from profiles server-side, and validate all request fields with strict length, action and MIME limits. Change WhatsApp input from raw phone to `customer_id`, fetch that customer only within the caller shop, and then send. Implement atomic user and shop rate limits with separate lower limits for image generation and WhatsApp. Return 401, 403, 400 and 429 correctly; keep CORS origin-configured; avoid raw provider errors and PII logs. Add automated tests covering unauthenticated, cross-shop, malformed, quota and valid requests. Do not put service-role keys or provider secrets in the frontend.

Repair shops RLS and prove tenant isolation

CRITICAL

CONFIRMED

Current code evidence

The initial migration enables RLS on shops but defines no SELECT or UPDATE policy for shops. `src/pages/Onboarding.tsx` reads `onboarding_completed` from shops and updates `name`, `shop_type` and `onboarding_completed` from the browser. Product image uploads also use the `product-images` bucket but this migration does not show storage-object policies.

Why this matters

With RLS enabled and no shops policy, normal client onboarding can fail. A weak policy written in haste can also permit cross-tenant reads or writes. Storage is a second tenant boundary and must match database isolation.

Implementation contract

- Create a new migration that adds explicit shops SELECT and UPDATE policies. A user may read or update only the `shop_id` on their own profile. Add WITH CHECK to prevent changing a row outside that relationship.
- Review profiles policies too. Prevent a normal client from changing role or `shop_id`. If profile creation is trigger-owned, do not add a broad client INSERT policy.
- Define `storage.objects` policies for the `product-images` bucket that require the first path segment to equal the caller `shop_id` and restrict allowed object operations to the caller tenant.
- Use storage paths that start with the known server-side shop id. Validate upload file type and byte size on the client and, where possible, in an upload Edge Function or storage policy flow.
- Harden the security-definer signup trigger with an explicit `search_path` and least-privilege grants. Keep shop/profile creation in the trigger rather than exposing broad table creation to clients.

Acceptance checks

- Account A can complete onboarding and later read/update only its own shop.
- Account B cannot SELECT, UPDATE, upload, list, overwrite or delete any A shop row or product-image object.
- A direct REST request attempting to set `profile.role` or `profile.shop_id` is rejected.
- Two-account automated integration tests pass against a clean staging database, including upload and retrieval of an image in each tenant path.

READY-TO-PASTE ANTIGRAVITY PROMPT

Create a new timestamped Supabase migration for RetailFlow tenant security. Do not edit an applied migration. Add least-privilege RLS policies for shops so a signed-in user can read and update only the shop referenced by their own profiles row, with matching USING and WITH CHECK clauses. Lock profiles so users cannot change role or `shop_id`. Add tenant-safe `storage.objects` policies for product-images using the first path component as `shop_id`. Review the signup security-definer trigger for an explicit `search_path` and minimum grants. Add two-account integration tests that verify onboarding, shop reads/updates, product image upload/list/read/delete, and cross-shop denial. Include rollback-safe migration comments and document any assumptions about the public bucket.

Replace dashboard fixtures with operational shop data

HIGH

CONFIRMED

Current code evidence

src/pages/Dashboard.tsx declares a fixed followUps array and fixed values 8, 12, INR 480000 and 6. The date is also a literal. None of these cards query Supabase, so the screen looks operational but does not reflect a showroom.

Why this matters

Owners can make sales decisions from inaccurate totals and follow-up lists. Hard-coded figures also make empty states, loading states and errors impossible to distinguish from real data.

Implementation contract

- Define the dashboard metrics precisely: active leads, follow-ups due, low-stock count, and open-quote value. Document which customer and quotation statuses are included.
- Fetch only records permitted by RLS and compute the metrics from the authenticated shop. Prefer a small SQL view or RPC returning the four aggregates and a follow-up list over downloading all rows to the browser.
- Add a real follow_up_at or next_action_at field if one does not exist. Do not infer urgency from created_at alone.
- Use local timezone-aware dates, loading skeletons, an empty state, retry/error feedback, and an Updated timestamp based on real query data.
- Make each follow-up row navigate to that exact customer and make metric cards link to a pre-filtered operational list.

Acceptance checks

- Changing records in shop A changes only shop A dashboard metrics after refresh/revalidation.
- An empty shop shows zero-state copy, never sample people or sample currency values.
- A follow-up due today, tomorrow and overdue are ordered and labelled correctly in the configured shop timezone.
- Metric definitions are covered by unit/integration tests and match the status pipeline selected in finding 6.

READY-TO-PASTE ANTIGRAVITY PROMPT

Replace the hard-coded RetailFlow dashboard with authenticated, tenant-scoped operational data. Define and document the exact SQL meaning of active leads, follow-ups due, low stock and open quote value. Add a safe next_action_at or follow_up_at field if needed, then provide a small RLS-respecting RPC or view that returns aggregates plus the priority follow-up list for the current shop. Update Dashboard.tsx with loading, empty and error states; use the shop timezone for due labels; link cards and rows to the relevant filtered customer/product/quote screens. Remove all sample names, dates and totals. Add tests for tenant isolation, zero data, overdue/today/tomorrow ordering and metric status inclusion.

Complete the persisted quotation-to-payment sales lifecycle

HIGH

PARTIALLY IMPLEMENTED

Current code evidence

src/pages/QuoteBuilder.tsx is now an interactive UI with local quantities, waste and totals, but it uses mock customer/product data and does not save to quotations or quotation_items. The schema has basic quotation tables but lacks a quote number, lifecycle status, immutable snapshots, approval/order/payment fields, PDF workflow and customer share record.

Why this matters

The current flow cannot be audited, reopened, approved, converted to an order or reconciled with payment. A visually correct quote can be lost on refresh and the dashboard cannot measure real pipeline value.

Implementation contract

- Create a versioned migration for quote_number, status, currency, tax/waste/transport/discount values, validity date, notes, created_by, sent_at, approved_at and immutable line-item snapshots such as product_name, SKU, unit, dimensions and unit_price.
- Adopt a minimal lifecycle: draft, sent, viewed, approved, expired, rejected, converted. Add order and payment tables only when needed, linked from the approved/converted quote with audit timestamps.
- Replace QuoteBuilder mock data with RLS-scoped customer and product selectors. Calculate totals deterministically in a shared domain function; validate them again server-side before persistence.
- Persist quote and items in one database transaction/RPC. Prevent a quote from referencing a product or customer from another shop and prevent edits to immutable sent snapshots without creating a revision.
- Generate a server-side PDF from a saved quote, store it in tenant-safe storage, and share a signed URL or tracked WhatsApp template. Do not expose a public mutable PDF URL.
- Add approval/order/payment screens only after their database contracts and status transitions are enforced. Record who performed each transition and when.

Acceptance checks

- A saved draft survives refresh and contains only the current shop customer/products/items.
- Cross-shop customer/product IDs are rejected by database/RPC tests.
- The displayed and persisted totals agree for waste, tax, transport and discount edge cases.
- A sent quote produces a downloadable tenant-authorized PDF and a recorded delivery event.
- Invalid lifecycle changes and edits to sent snapshots are rejected or create an explicit new revision.

READY-TO-PASTE ANTIGRAVITY PROMPT

Turn the existing RetailFlow QuoteBuilder UI into a real quotation lifecycle without discarding its current UX. Add a new Supabase migration for quote number, currency, lifecycle status, validity, audit timestamps and immutable item snapshots. Use a single tenant-safe RPC/transaction to create or update draft quotations and quotation_items after validating all customer/product IDs against the caller shop. Replace mock lists with real RLS-scoped data. Put total calculations in a shared tested domain module and validate server-side. Implement draft, sent, viewed, approved, expired, rejected and converted transitions with audit history. Generate PDFs only from saved quotes, keep them in tenant-safe storage, and share signed links or a tracked WhatsApp template. Add tests for refresh persistence, cross-shop denial, total calculations, state transitions and authorized PDF access.

Finish or deliberately hide incomplete UI controls

HIGH

MIXED

Current code evidence

The refreshed Products screen has working search and filter state, and Customers has working status filtering. However, `src/pages/More.tsx` renders clickable settings rows with no actions. `src/components/ui/ProductCard.tsx` shows a three-dot `MoreHorizontal` icon without an action, while its `onFavorite` prop exists but is unused. `QuoteBuilder` also has a visible quote-options ellipsis with no behavior.

Why this matters

A button that appears available but does nothing reduces user trust and makes support harder. It is better to ship a clearly scoped feature or remove the affordance until its workflow is ready.

Implementation contract

- Do not regress the working product/customer filters. Keep their current functional state and cover it with a focused UI test.
- For every visible control, choose one: complete a useful workflow, show an explicit disabled/coming-soon state with no false affordance, or remove it from the production UI.
- Implement `More` rows only when they lead to real routes/dialogs and persist changes. At minimum, keep `Sign out` working; do not expose privacy/team settings that do nothing.
- Replace the product three-dot icon with an accessible menu containing only implemented actions, or remove it. Either implement favourite persistence with a schema/RLS contract or delete the unused prop.
- Give `QuoteBuilder` options a meaningful menu (duplicate, discard draft, print preview) only after the quote persistence work exists; otherwise remove the ellipsis.

Acceptance checks

- Every visible button has a deterministic outcome, browser-accessible name and keyboard focus path.
- No click handler is empty and no icon communicates an unavailable action without an explanatory state.
- A test verifies product/category/status filtering, `More` navigation or disabled states, product menu behavior, and `Sign out`.

READY-TO-PASTE ANTIGRAVITY PROMPT

Audit every visible interactive control in `RetailFlow` and remove false affordances. Preserve the already working `Products` and `Customers` filters. For `More`, `ProductCard` and `QuoteBuilder` options, either implement a complete accessible workflow with real routes/persistence or remove/disable the control with clear explanatory copy. Replace the product three-dot icon with a labelled menu only if it has implemented items; either build favourite persistence with a migration and RLS or remove the unused `onFavorite` API. Ensure all controls have accessible names, keyboard operation, loading/error states where relevant, and focused UI tests. Do not introduce placeholder pages or no-op click handlers.

Standardise the customer sales pipeline

MEDIUM	CONFIRMED
--------	-----------

Current code evidence

Customers.tsx creates and filters new, follow_up, converted and lost. CustomerCard.tsx additionally defines quoted and won. The database visit_status is unrestricted text. Dashboard, quote lifecycle and analytics cannot reliably agree on which status counts as an active lead or a win.

Why this matters

Status drift produces incorrect dashboards, confusing labels and unsafe migrations. A lead can be labelled converted in one screen and won in another, making reports and automations ambiguous.

Implementation contract

- Adopt one canonical lead pipeline: new, follow_up, quoted, won, lost. Treat converted as a legacy value mapped to won during migration; do not keep both as active concepts.
- Add a database CHECK constraint or enum, a data migration mapping legacy values, and a safe default. Audit any existing null or unexpected values before enforcing the constraint.
- Update the customer form, CustomerCard, filters, translations, AI context and dashboard definitions to use exactly the same constants from a shared TypeScript module.
- Keep quote status separate from customer lead status. Sending a quote can set the customer to quoted through an explicit business rule, not through card rendering.
- Add next_action_at/follow_up_at for follow-ups instead of using status alone as a reminder system.

Acceptance checks

- The database rejects an invalid status and contains no converted records after migration.
- Every status available in the customer form has a defined card label, filter, translation and dashboard behaviour.
- Quote sent/approved actions update customer status only through tested explicit transitions.

READY-TO-PASTE ANTIGRAVITY PROMPT

Standardise RetailFlow customer status values across database and UI. Use the canonical pipeline new, follow_up, quoted, won and lost. Create a new migration that audits/migrates converted to won, applies a safe default and adds a CHECK constraint or enum. Create one shared TypeScript status definition used by Customers, CustomerCard, filters, translations, AI context and dashboard metrics. Keep quotation status separate from customer status and only transition a customer through explicit tested actions. Add a next_action_at or follow_up_at field for reminders. Include migration tests, UI status coverage and dashboard metric tests.

Finish login recovery, password visibility and OAuth feedback

MEDIUM

PARTIALLY RESOLVED

Current code evidence

src/pages/Login.tsx supports password sign-in, sign-up and Google OAuth, but lacks forgot-password and show/hide-password controls. The Google catch block only writes to console, so an unexpected OAuth failure gives no user feedback. The previous audit called sign-up success red; the current code already renders success in a green success panel and should retain that improvement.

Why this matters

Auth failures are high-friction. Without recovery and meaningful feedback, users may abandon the app or repeatedly create accounts. Error messages must remain helpful without exposing account-enumeration details.

Implementation contract

- Add a forgot-password path using Supabase resetPasswordForEmail with a configured redirect route, generic success copy, resend protection and loading/error feedback.
- Add a labelled show/hide password toggle that works with keyboard and screen readers. Preserve autocomplete attributes and do not log passwords.
- Handle both OAuth response errors and thrown exceptions by setting a friendly error state. Include a retry option and a safe explanation for popup/redirect failure.
- Keep successful sign-up/reset messages in the current green success style. Use a red panel only for actual errors and clear stale messages on mode switch.
- Implement the reset-password completion route and require a new password plus confirmation before updateUser is called.

Acceptance checks

- Forgot password does not reveal whether an email account exists and shows a generic confirmation message.
- Password visibility can be toggled by mouse and keyboard with correct aria-label text.
- OAuth errors are visible to the user, not only console logs; normal OAuth redirects still work.
- Sign-up and reset success panels are green; authentication failures are red and actionable.

READY-TO-PASTE ANTIGRAVITY PROMPT

Improve RetailFlow authentication without changing the current green sign-up success treatment. Add forgot-password using Supabase resetPasswordForEmail, a configured reset route that verifies a new password and confirmation, generic anti-enumeration copy, resend/loading/error handling, and no sensitive logging. Add an accessible show/hide password button that preserves autocomplete. Make Google OAuth errors visible for both returned errors and thrown exceptions, with a retry path. Clear stale feedback when switching sign-in/sign-up mode. Add focused tests for reset success/error, password visibility, OAuth failure feedback and message styling.

Remove broken encoding and make INR presentation consistent

MEDIUM

CONFIRMED

Current code evidence

Current source contains mojibake such as 'Â', 'Ã—' and 'â,' in CustomerCard.tsx, ProductCard.tsx and AddProductModal.tsx. The product form appears intended to show INR but renders a broken rupee literal. The UI otherwise uses Intl.NumberFormat with INR and Lucide IndianRupee, so the presentation contract is inconsistent.

Why this matters

Broken text is visible to customers and reduces confidence. Currency ambiguity can cause commercial mistakes, especially when a product field, quote and dashboard use different labels.

Implementation contract

- Save TypeScript/TSX/JSON files as UTF-8 and replace mojibake literals with correct source text. Use normal text separators or Lucide icons instead of copied symbols when they are more robust.
- Create one INR formatting helper using Intl.NumberFormat('en-IN', { currency: 'INR' }) and reuse it for product, dashboard and quote money values. Clarify the unit: per sq ft, per piece, or total.
- Rename the product label to an explicit format such as 'Price (INR per sq ft)' based on the actual data contract, rather than a bare dollar or broken rupee character.
- Run a repository scan for common mojibake patterns and fix user-facing source strings, translations and generated templates. Do not alter binary assets or encoded base64 data.
- Use Lucide icons for visual currency/action symbols where appropriate and ensure fallback text remains understandable to screen readers.

Acceptance checks

- Repository text scan finds no remaining user-facing â, Ã or Â mojibake fragments.
- Product forms, cards, quote totals and dashboard currency values all render INR correctly in a browser.
- A product price label states its unit and agrees with quote calculation tests.

READY-TO-PASTE ANTIGRAVITY PROMPT

Normalise RetailFlow text encoding and monetary UI. Treat all source and locale files as UTF-8, scan for user-facing mojibake such as â, Ã and Â, and replace only text literals with correct Unicode-safe text or Lucide icons. Do not touch binary assets or base64. Create one shared INR formatting helper based on Intl.NumberFormat and reuse it in products, quote totals and dashboard. Change the product price label to an explicit INR label with the real unit, for example INR per sq ft or INR per piece. Add browser/render tests and a source scan regression check so broken characters cannot return.

Minimise AI data, get consent and bound uploads

MEDIUM

CONFIRMED

Current code evidence

src/pages/AI.tsx constructs the first chat prompt from the full shop catalogue and customer list. The customer portion includes name, phone, budget, needs, status and date, then sends it to gemini-proxy. AI image inputs can be sent as base64 image parts, while the client does not enforce a clear byte limit before readAsDataURL.

Why this matters

Phone numbers and free-form notes can be personal data. Sending all customers when a user asks a general question is unnecessary disclosure. Large images also increase cost, latency and denial-of-service exposure.

Implementation contract

- Remove phone numbers and unneeded free-form personal data from AI prompts. Default to aggregated or minimised context and fetch only the smallest relevant records for a specific question.
- Show a clear in-product disclosure before first AI use: what information is sent to the AI provider, purpose, user choice, provider retention link/policy, and an opt-out or consent setting stored per shop/user.
- Create a server-side AI context builder that derives tenant-scoped, minimised records after authentication. Do not allow the client to submit a raw SYSTEM CONTEXT string.
- Set file count, MIME, decoded byte, pixel-dimension and rate limits on both client and Edge Function. Resize/compress images before inference and reject non-image payloads.
- Set a retention/deletion policy for AI audit metadata and uploaded/generated assets. Log only request type, size, status and cost-safe identifiers, not prompt bodies or PII.

Acceptance checks

- Prompt inspection proves customer phone numbers are never sent to Gemini for normal chat.
- AI cannot be used until the required disclosure/consent state is recorded, unless the configured policy allows a non-personal-data mode.
- Oversized, invalid MIME and excessive image requests are rejected before provider invocation on both client and server paths.
- A user in shop A can never obtain contextual customer/product data from shop B through AI.

READY-TO-PASTE ANTIGRAVITY PROMPT

Make RetailFlow AI privacy-first. Remove customer phone numbers and unnecessary free-form PII from Gemini inputs. Move context construction to the authenticated Edge Function so it derives only tenant-scoped, minimum-needed product/customer fields and the browser cannot inject raw system context. Add a first-use AI disclosure and stored consent/opt-out setting explaining what is sent, why and retention. Enforce file count, MIME, decoded bytes, dimensions and rate limits on client and server; resize/compress before inference. Define retention/deletion for AI audit metadata and generated assets, and never log prompts, phones, tokens or provider secrets. Add tests proving no phone transfer, cross-shop isolation, consent gating and upload rejection before provider calls.

Definition of done for the remediation branch

- All Critical findings are deployed to staging and pass unauthenticated, cross-tenant and rate-limit tests before any public release.
- Every data migration is new, reversible where practical, documented, and tested with two independent user accounts.
- The quotation lifecycle persists real data, has enforced tenant scope and only shares tenant-authorized PDFs.
- The dashboard contains no sample people, sample totals or fixed dates; all user-visible controls either work or are intentionally unavailable.
- No user-facing mojibake remains, login feedback is accessible, and AI has a minimal-data consent path plus upload limits.
- npm run build, lint, database migration tests, Edge Function tests and focused UI tests pass. Record remaining pre-existing warnings separately from new work.

Handoff note for Antigravity

Start with findings 1 and 2. Do not compensate for missing server security with client-side checks. Preserve the current UI refresh and the working product/customer filters while replacing mocked data with tenant-scoped data.